Parul University

FACULTY OF ENGINEERING AND TECHNOLOGY

BACHELOR OF TECHNOLOGY

ENTERPRISE PROGRAMMING USING JAVA LABORATORY

(303105310)

SEMESTER V

Computer Science & Engineering Department

Laboratory Manual

Session 2026-27

Faculty of Engineering & Technology
Subject: Enterprise Prog. using Java
Subject Code: 303105310
B.Tech. CSE Year 3 Semester 5

CERTIFICATE

This is to certify that Mr. **Ajeet Kumar** with enrolment no. 2403051050019 has successfully completed his laboratory experiments in the **ENTERPRISE PROG. USING JAVA – LAB (303105310)** from the department of **COMPUTER SCIENCE & ENGINEERING** during the academic year 2026-2027.

योगः कर्मसु कौशलम्
PARUL UNIVERSITY

Date of Submission:..... Staff In charge:.....

Head Of Department:.....

INDEX

Sr. No.	Experiment Title	Page No.		Date		Mark	Sign
		From	To	Performance	Assessment		
1.	Write a program to insert and retrieve the data from database using JDBC.						
2.	Write a program to demonstrate the use of Prepared Statement and Result Set interface						
3.	Servlet Programming Servlet Execution on tomcat A servlet program to print hello world A servlet program to display request details A servlet program to handle user form A servlet program to create a cookie A servlet program to display cookie A servlet program to do session tracking Write a program to implement chat Server using Server Socket and Socket class. Write a Servlet program to send username and password using HTML forms and authenticate the user						
4.	JSP Programming JSP program to display hello world. JSP program to demonstrate arithmetic operations JSP program to demonstrate jsp: forward action tag JSP program to request implicit object Developing a web application to insert record into Oracle Database using JSP and JDBC						
5.	Create application to store the data in database to perform Hibernate CRUD operations.						
6.	Create a application store the data in database to perform Spring CRUD operations.						
7.	Create a web application to store the data in database with spring boot.						

Practical – 1

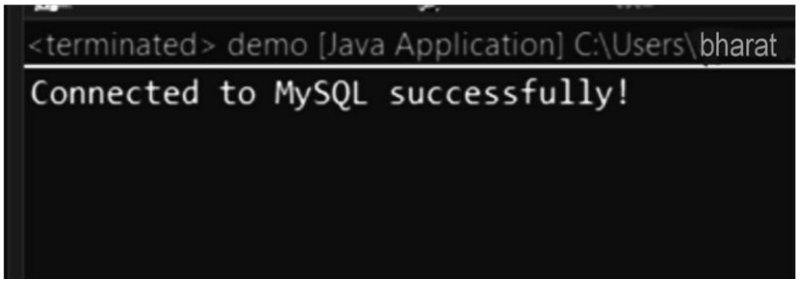
Aim - Write a program to insert and retrieve the data from database using JDBC.

CODE:

```
package jdbcconnect;
import
java.sql.Connection;
import
java.sql.DriverManager;
import
java.sql.SQLException;
public class sample {
  public static void main(String[]
    args) { Connection conn =
    null;
    try {
Class.forName("com.mysql.cj.jdbc.D
river"); conn =
DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/testdb", "root",
      "root"); System.out.println("Connected to MySQL
      successfully!");
    } catch (Exception e) {
      e.printStackTrace();
    } finally {
      try {
        if (conn != null)
          conn.close();
```

```
            } catch (SQLException ex) {
                ex.printStackTrace();
            }
        }
    }
}
```

OUTPUT :

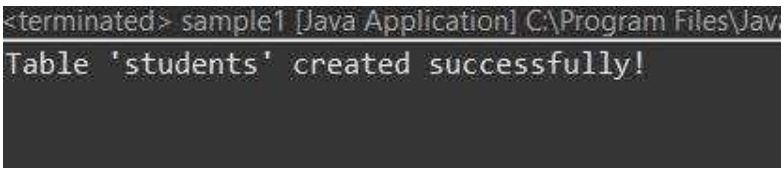


CODE:

```
package demo;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.SQLException;
public class sample1 {
  public static void main(String[] args) {
    Connection conn = null;
    Statement stmt =
    null; try {
      // Load the MySQL JDBC Driver
      Class.forName("com.mysql.cj.jdbc.Driver");
      // Connect to the database
      conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/testdb", "root",
        "root");
      // SQL to create table
      String sql = "CREATE TABLE students (" +
            "id INT PRIMARY KEY
            AUTO_INCREMENT," + "name
            VARCHAR(100)," +
            "age INT," +
            "course VARCHAR(50))";
      // Create and execute
      statement stmt =
      conn.createStatement();
      stmt.executeUpdate(sql);
      System.out.println("Table 'students' created successfully!");
    } catch (Exception e) {
      e.printStackTrace();
    } finally {
      try {
        if (stmt != null)
        stmt.close(); if (conn !=
        null) conn.close();
      } catch (SQLException
        ex) {
        ex.printStackTrace();
      }
    }
  }
}
```

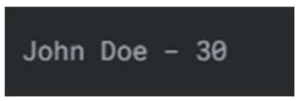
OUTPUT:




```
Package demo
// Example JDBC Code
import java.sql.*;

public class JDBCExample {
        public static void main(String[] args) {
            try {
            Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb",
   "root", "password");
            Statement stmt = conn.createStatement();
            stmt.executeUpdate("INSERT INTO users (name, age) VALUES ('John Doe', 30)");
                ResultSet rs = stmt.executeQuery("SELECT * FROM users");
                while (rs.next()) {
                        System.out.println(rs.getString("name") + " - " + rs.getInt("age"));
                 }
                conn.close();
         } catch (Exception e) {
                e.printStackTrace();
        }
    }
}
```

Output:



Practical – 2

Aim - Write a program to demonstrate the use of Prepared Statement and Result Set interface

CODE:

```
package demo;


import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.PreparedStatement;
import java.util.*;
public class App {
    public static void main(String[] args) {
        Connection conn = null;
        PreparedStatement pstmt = null;

        String name,course;
        int age;

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter Name :");

        name=sc.nextLine();
        System.out.print("Enter Age :");

        age=sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Course :");

        course=sc.nextLine();
```

```
        try {
            // Step 1: Load JDBC driver
            //Class.forName("com.mysql.cj.jdbc.Driver");

            // Step 2: Connect to MySQL database
            conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb", "root",
"bharat@999");

            String sql="insert into student(name,age,course) values(?,?,?)";

            pstmt = conn.prepareStatement(sql);

            pstmt.setString(1, name);
            pstmt.setInt(2, age);
            pstmt.setString(3, course);


            int rowsInserted = pstmt.executeUpdate();

            if (rowsInserted > 0) {
                System.out.println("Record inserted successfully!");
            }

            // Step 3: Confirmation
            System.out.println("Connected to MySQL successfully!");

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            try {
```

```
            if (pstmt != null) pstmt.close();
              if (conn != null)
                  conn.close();
            } catch (SQLException ex) {
              ex.printStackTrace();
            }
        }
    }
}
```

OUTPUT:

```
Enter Name :Bharat
Enter Age :24
Enter Course :CSE
Record inserted successfully!
Connected to MySQL successfully!
```

CODE:

```
package demo;


import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.PreparedStatement;
import java.util.*;
import java.sql.ResultSet;
public class JDBCEx {
    public static void main(String[] args) {
        Connection conn = null;
        PreparedStatement pstmt = null;
        ResultSet rs = null;

        try {

            conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb", "root",
"bharat@999");

            String sql="Select * from student;";

            pstmt = conn.prepareStatement(sql);

            rs = pstmt.executeQuery();

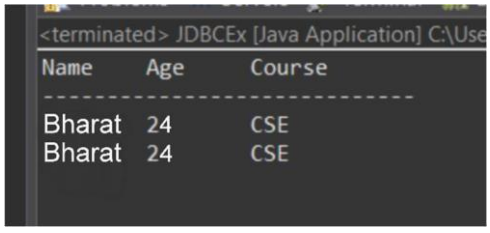
            System.out.println("Name\tAge\tCourse");
            System.out.println(" ---------------------------------");
            while (rs.next()) {
                String name = rs.getString("name");
```

```
            int age = rs.getInt("age");
            String course = rs.getString("course");
            System.out.println(name + "\t" + age + "\t" + course);

        }

    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        try {
          if (pstmt != null) pstmt.close();
            if (conn != null)
                conn.close();
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
  }
}
```

OUTPUT:



Practical – 3

Aim - Servlet Programming Servlet Execution on tomcat A servlet program to print hello world A servlet program to display request details A servlet program to handle user form A servlet program to create a cookie A servlet program to display cookie A servlet program to do session tracking Write a program to implement chat Server using Server Socket and Socket class. Write a Servlet program to send username and password using HTML forms and authenticate the user.

Steps to Configure Servlet :

1. In Eclipse: File → New → Maven Project.
2. Check Create a simple project → Next.
3. Fill details:
4. Group Id: com.example
5. Artifact Id: HelloServletApp
6. Packaging: war
7. Version: 1.0-SNAPSHOT
8. Finish → Maven project is created.
9. In Eclipse: Window → Show View → Servers .
10. In Servers view → New → Server.
11. Choose Apache → Tomcat 10.36 Server.
12. Set the Tomcat installation directory and click Finish.

Run on Server :

1. Right-click project → Run As → Run on Server.
2. Choose your Tomcat server and Finish.
3. Open a browser and go to:
4. http://localhost:8080/HelloServletApp/hello

- **web.xml:**

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
        version="3.1">

  <servlet>
    <servlet-name>HelloServlet</servlet-name>
    <servlet-class>com.example.HelloServlet</servlet-class>
  </servlet>
```

```
  <servlet-mapping>
    <servlet-name>HelloServlet</servlet-name>
    <url-pattern>/hello</url-pattern>
  </servlet-mapping>

</web-app>
```

- **pom.xml**

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>ServletApp</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>war</packaging>
  <dependencies>
  <!-- https://mvnrepository.com/artifact/jakarta.servlet/jakarta.servlet-api  -->
<dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.1.0</version>
    <scope>provided</scope>
</dependency>
</dependencies>
</project>
```

- **First Servlet Code**

CODE:

```
package com.example; import java.io.IOException;

import java.io.PrintWriter;

import jakarta.servlet.ServletException;

import jakarta.servlet.http.HttpServlet;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;


public class HelloServlet extends HttpServlet

{

  private static final long serialVersionUID = 1L;


  @Override

  protected void doGet(HttpServletRequest request, HttpServletResponse

      response) throws ServletException, IOException

  {

    response.setContentType("text/h

    tml");    PrintWriter    out    =

    response.getWriter();

    out.println("<html><body>");

    out.println("<h2>Hello, this is your first servlet!</h2>");

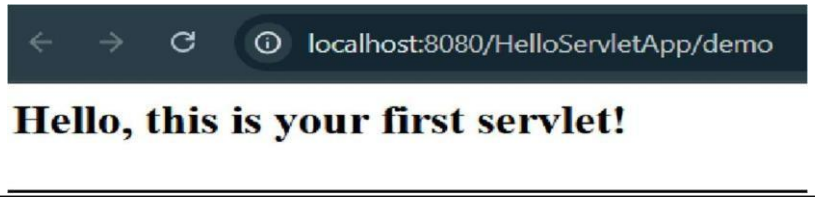
    out.println("</body></html>");

    out.close();

  }

}
```

OUTPUT:



- **Static Cookie Code**

CODE:

```
package com.example;
import
java.io.IOException;
import
java.io.PrintWriter;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/CookieDemo")
public class cookie extends HttpServlet {
  /**
         *
         */
        private static final long serialVersionUID = 1L;

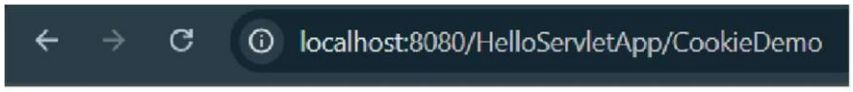
         protected void doGet(HttpServletRequest request, HttpServletResponse
       response) throws ServletException, IOException {

    response.setContentType("text/html"
    ); PrintWriter out =
```

```
        response.getWriter();
        // Write a cookie
        Cookie c = new Cookie("username",
        "CodeTantra"); c.setMaxAge(3600); // 1
        hour response.addCookie(c);

        // Read cookies
        Cookie[] cookies =
        request.getCookies(); if (cookies
        != null) {
          for (Cookie ck : cookies) {
            if (ck.getName().equals("username")) {
              out.println("Hello, " + ck.getValue());
            }
          }
        } else {
          out.println("No cookies found.");
        }
      }
}
```

OUTPUT:



Hello, CodeTantra

- **Dynamic Cookie Code**

CODE:

```
package com.example;

import java.io.*;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/Form")
public class Createcookies extends HttpServlet {
  /**
         *
         */
        private static final long serialVersionUID = 1L;

        protected void doGet(HttpServletRequest request, HttpServletResponse
    response) throws ServletException, IOException {

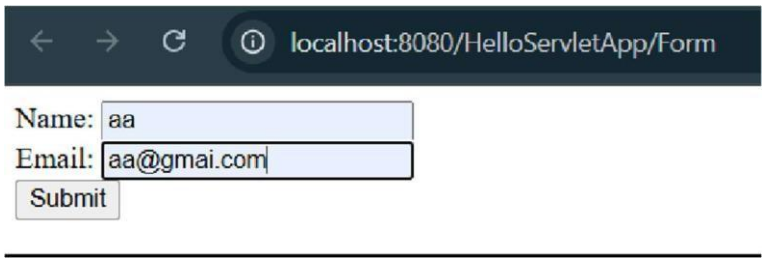
    response.setContentType("text/html"
    ); PrintWriter out =
    response.getWriter();

    out.println("<form action=\"FormHandler\" method=\"post\">\r\n"
```

```
        + "  Name: <CODEtype=\"text\" name=\"username\"><br>\r\n"
        + "  Email: <CODEtype=\"email\" name=\"email\"><br>\r\n"
        + "  <CODEtype=\"submit\" value=\"Submit\">\r\n"
        + "</form>");


    }
}
```

OUTPUT:



CODE:

```
package com.example;
import java.io.*;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/FormHandler")
public class Readform extends HttpServlet {
/**
         *
         */
        private static final long serialVersionUID = 1L;
protected void doPost(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
response.setContentType("text/html");
PrintWriter out = response.getWriter();
String name =
request.getParameter("username");
String email =
request.getParameter("email"); Cookie c
= new Cookie("username", name);
Cookie c1=new Cookie("Email",email);
c.setMaxAge(60); // 60 second
c1.setMaxAge(60);
```

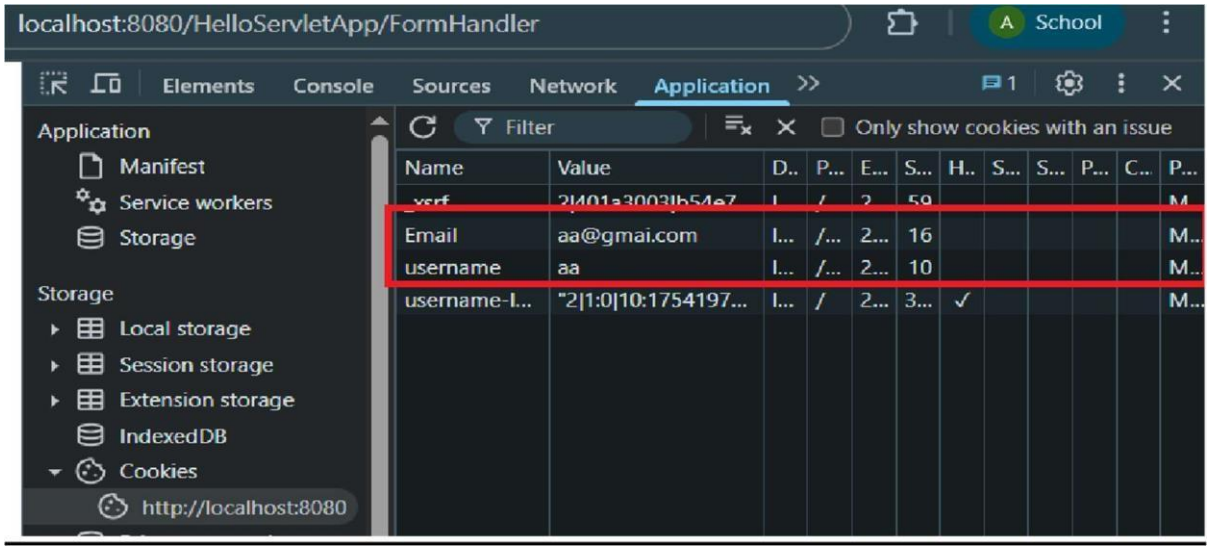
```
response.addCookie(c);

response.addCookie(c1);

}

}
```

OUTPUT:



Practical – 4

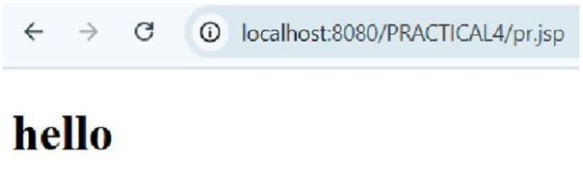
Aim - JSP Programming JSP program to display hello world. JSP program to demonstrate arithmetic operations JSP program to demonstrate jsp: forward action tag JSP program to request implicit object Developing a web application to insert record into Oracle Database using JSP and JDBC

- **hello.jsp**

CODE:

```
<%@ page language="java" contentType="text/html;
   charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>hello world</title>
</head>
<body>
<h1>hello</h1>
</body>
</html>
```

OUTPUT:



- **addition.jsp**

CODE:

```
<%@ page  language="java" contentType="text/html" pageEncoding="UTF-8" %>

<!DOCTYPE HTML>

<html>

	<head>

	</head>

	<body>

		<h2> Hello World </h2>

			<%
			int a =10;
			int b = 20;
			int c = 0;
		%>


	Add : <%= a+b+c %><br/>

		Sub : <%= a-b-c %><br/>
		Mul : <%= a*b*c %><br/>
		Div : <%= a/b %><br/>
		Mod : <%= a%b %><br/>



		<% for(int i=0; i< 10; i++){ %>

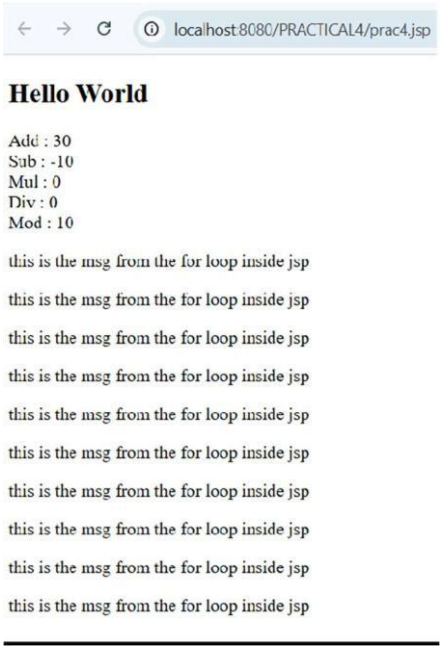
			<p> this is the msg from the for loop inside jsp </p>

		<% } %>

	</body>

</html>
```

OUTPUT:

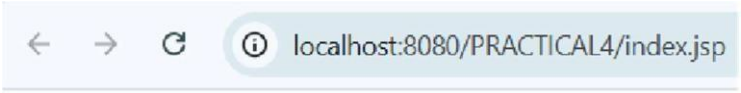

localhost:8080/PRACTICAL4/prac4.jsp
Hello World
Add : 30
Sub : -10
Mul : 0
Div : 0
Mod : 10
this is the msg from the for loop inside jsp
this is the msg from the for loop inside jsp
this is the msg from the for loop inside jsp
this is the msg from the for loop inside jsp
this is the msg from the for loop inside jsp
this is the msg from the for loop inside jsp
this is the msg from the for loop inside jsp
this is the msg from the for loop inside jsp
this is the msg from the for loop inside jsp
this is the msg from the for loop inside jsp

- **index.jsp**

CODE:

```
<%@ page language="java" contentType="text/html;
   charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<body>
   <h3>This is index.jsp</h3>
   <jsp     : forward page = “Welcome.jsp”/>
</body>
</html>
```

OUTPUT:



Welcome to jsp forward example

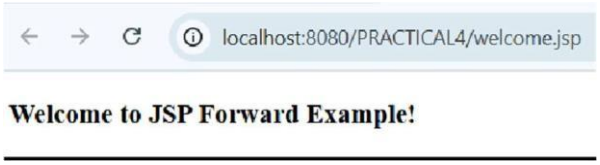
- **welcome.jsp**

CODE:

```
<%@ page language="java" contentType="text/html;
   charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<body>
   <h3>Welcome to JSP Forward Example!</h3>
</body>
```

```
</html>
```

OUTPUT:


localhost:8080/PRACTICAL4/welcome.jsp
Welcome to JSP Forward Example!

Practical – 5

Aim - Create application to store the data in database to perform Hibernate CRUD operations.

- **MainApp.java**

CODE:

```
package hibernatecrud;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
public class MainApp {
public static void main(String[] args) {
// Build SessionFactory (reads hibernate.cfg.xml)
Configuration config = new Configuration().configure();
try (SessionFactory sf = config.buildSessionFactory()) {
// --- CREATE ---
try (Session session = sf.openSession()) {
Transaction tx = session.beginTransaction();
Student s = new Student();
s.setId(102);
s.setName("Ashish");
s.setMarks(79);
session.persist(s); // insert
tx.commit();
System.out.println("Student created successfully!");
}
// --- READ ---
 try (Session session = sf.openSession()) {
 Student s = session.find(Student.class, 101); // fetch by ID
```

```
if (s != null) {
System.out.println("\n--- Student Details ---");
System.out.println("ID: " + s.getId());
System.out.println("Name: " + s.getName());
System.out.println("Marks: " + s.getMarks());
} else {
System.out.println("Student not found!");
}
}
// --- UPDATE ---
try (Session session = sf.openSession()) {
Transaction tx = session.beginTransaction();

Student s = session.find(Student.class, 101);
if (s != null) {
s.setName("Susruthi");
s.setMarks(95);
session.merge(s); // update
tx.commit();
System.out.println("\nStudent updated successfully!");
} else {

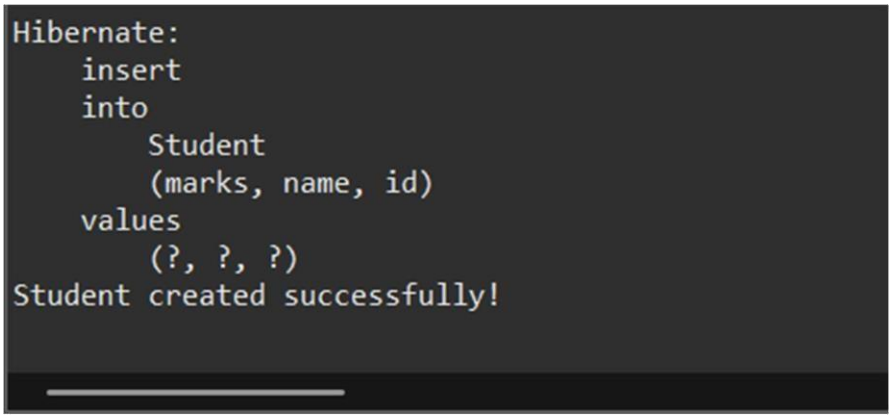
System.out.println("Student not found for update!");
}
}
// --- READ AFTER UPDATE ---
try (Session session = sf.openSession()) {
Student s = session.find(Student.class, 101);
```

```
if (s != null) {
System.out.println("\n--- Updated Student Details ---");
System.out.println("ID: " + s.getId());
System.out.println("Name: " + s.getName());
System.out.println("Marks: " + s.getMarks());
}
}
// --- DELETE ---
try (Session session = sf.openSession()) {
Transaction tx = session.beginTransaction();

Student s = session.find(Student.class, 101);
if (s != null) {
session.remove(s);
tx.commit();
System.out.println("\nStudent deleted successfully!");
} else {
System.out.println("Student not found for deletion!");
}
}
} catch (Exception e) {
e.printStackTrace();
}

}
}
```

OUTPUT:


Hibernate:
insert
into
Student
(marks, name, id)
values
(?, ?, ?)
Student created successfully!

- **Student.java**

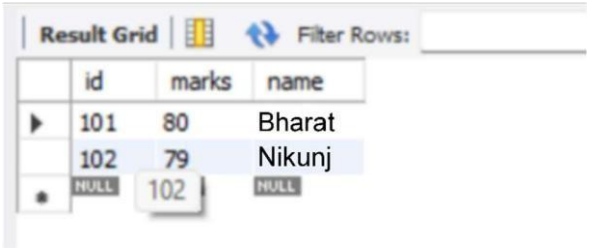
  CODE :

```
package hibernatecrud;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
@Entity
public class Student {
public Student() {
super();
}
@Id
private int id;
private String name;
private int marks;

public Student(int id, String name, int marks) {
super();
this.id = id;
this.name = name;
this.marks = marks;
}
// Getters and setters
public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public int getMarks() { return marks; }
public void setMarks(int marks) { this.marks = marks; }
```

```
@Override
public String toString() {
return "Student [id=" + id + ", name=" + name + ", marks=" + marks + "]";
}
}
```

OUTPUT :



- **Hibernate.cfg.xml :**

```
<?xml version="1.0" encoding="UTF-8"?>
<hibernate-configuration>
<session-factory>
<!-- Database connection settings -->
<property   name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
<property   name="hibernate.connection.url">jdbc:mysql://localhost:3306/test</property>
<property name="hibernate.connection.username">root</property>
<property name="hibernate.connection.password">root</property>

<!-- SQL dialect for MySQL 8 -->
<property   name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
<!-- Show generated SQL in console -->
<property name="hibernate.show_sql">true</property>
<property name="hibernate.format_sql">true</property>
<!-- Automatically create/update tables -->
<property name="hibernate.hbm2ddl.auto">update</property>
<!-- Disable second-level cache -->
<property  name="hibernate.cache.use_second_level_cache">false</property>
<!-- Annotated entity classes -->
<mapping class="hibernatecrud.Student"/>
</session-factory>
</hibernate-configuration>
```

Practical – 6

Aim - Create an application store the data in database to perform Spring CRUD operations.

- **Simple Spring Code :**

Car.java

```
package springcrudapp;


import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
@Component
class Car {
private Engine engine;
@Autowired
public Engine getEngine() {
return engine;
}
public void setEngine(Engine engine) {
this.engine = engine;
}
public void drive() {
engine.start();
System.out.println("Car is driving...");
}
}
```

Engine.java

```
package springcrudapp;
import org.springframework.stereotype.Component;
@Component
class Engine {
```

```
public void start() {
System.out.println("Engine started!");
}
}
```

MainSpring.java

```
package springcrudapp;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class MainSpring {
public static void main(String[] args) {
// TODO Auto-generated method stub
ApplicationContext ac=new

ClassPathXmlApplicationContext("springdemo1.xml");
Car c1=ac.getBean(Car.class);
c1.drive();
}
}
```

Springdemo1.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
      http://www.springframework.org/schema/beans
      http://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="epjcar" class="springcrudapp.Car">
```

```
    <property name="engine" ref="epjengine"/>
  </bean>

  <bean id="epjengine" class="springcrudapp.Engine"/>
</beans>
```

Outputs:



- **Spring CRUD Operations (ORM)**
- **Package : com.dao**

StudentDao.java

```
package com.dao;

import java.util.Date;

import java.util.List;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.springframework.beans.factory.annotation.Autowired; import
org.springframework.stereotype.Repository;

import org.springframework.transaction.annotation.Transactional; import com.pojo.Student;

@Repository

public class StudentDao {

private SessionFactory factory;

public StudentDao() {

super();

}

@Autowired

public void setFactory(SessionFactory factory) {

this.factory = factory;

}

// Retrieve entire data

@Transactional

public List<Student> find() {

Session session = factory.getCurrentSession();

List<Student> data = session.createQuery("From Student",Student.class).list();

System.out.println("data fetched successfully");

return data;
```

```
}
// Retrieve data by id
@Transactional
public Student find(Integer id) {
Session session = factory.getCurrentSession();
Student s = session.get(Student.class, id);
System.out.println("data fetched successfully");
return s;
}
// Insert data
@Transactional
public void insert(String name, long enroll_no, Date date, String dept) { Session session =
factory.getCurrentSession();
Student record = new Student(name,enroll_no, date, dept); session.persist(record);
System.out.println("data inserted successfully");
}
// update data
@Transactional
public void update(Student s) {
Session session = factory.getCurrentSession();
session.merge(s);
System.out.println("data updated successfully");
}
// remove data
@Transactional
public void remove(Student s) {
Session session = factory.getCurrentSession();
session.remove(s);
```

```
System.out.println("data removed successfully");
}
}
```

- **Package : com.pojo**

  Student.java

```
package com.pojo;
import java.util.Date;
import jakarta.persistence.Entity;
import   jakarta.persistence.GeneratedValue;
import   jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
@Entity
public class Student {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY) Integer id;
String name;
long enrollment_no;
Date dob;
String  department;
public  Student()  {
super();
// TODO Auto-generated constructor stub
}
public Student(String name, long enrollment_no, Date dob, String department) {
super();
this.name = name;
this.enrollment_no = enrollment_no;
this.dob = dob;
```

```
this.department = department;
}
public Integer getId() {
return id;
}
public void setId(Integer id) {
this.id = id;
}
public String getName() {
return name;
}
public void setName(String name) {
this.name = name;
}
public long getEnrollment_no() {
return enrollment_no;
}
public void setEnrollment_no(long enrollment_no) { this.enrollment_no = enrollment_no;
}
public Date getDob() {
return dob;
}
public void setDob(Date dob) {
this.dob = dob;
}
public String getDepartment() {
return department;
}
```

```
public void setDepartment(String department) { this.department = department;
}
}
```

- **Package : com.spring**

  SpringApplication.java

```
package com.spring;
import java.util.Date;
import java.util.List;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.dao.StudentDao;
import com.pojo.Student;
public class SpringApplication {
public static void main(String[] args) {
@SuppressWarnings("resource")
ApplicationContext container = new
ClassPathXmlApplicationContext("beans.xml");
StudentDao dao = container.getBean(StudentDao.class);
// insert single record
dao.insert("Ajay", 230510500, new Date(), "CSE");
// insert 10 records
for(int i = 0; i < 10; i++) {
dao.insert("name " + i, (long)(230510500 + i), new
Date(), "CSE");
}
// Retrieve and print record
List<Student> data = dao.find();
System.out.println();
```

```
for(int i = 0; i < data.size(); i++) {
System.out.print(data.get(i).getId() + " | ");
System.out.print(data.get(i).getName() + " | ");
System.out.print(data.get(i).getEnrollment_no() + " | ");
System.out.print(data.get(i).getDob() + " | ");
System.out.print(data.get(i).getDepartment()+"\n");
}
System.out.println();
// update 1st record
Student s = data.getFirst();
s.setDepartment("AIML");
dao.update(s);
// print the updated row
s = dao.find(s.getId());
System.out.print("\nupdated details : \n" + s.getId() + " | "); System.out.print(s.getName() + "
| ");
System.out.print(s.getEnrollment_no() + " | ");
System.out.print(s.getDob() + " | ");
System.out.print(s.getDepartment()+"\n");
// delete last row
s = data.getLast();
System.out.println("\nDeleted Record details : ");
System.out.print(s.getId() + " | ");
System.out.print(s.getName() + " | ");
System.out.print(s.getEnrollment_no() + " | ");
System.out.print(s.getDob() + " | ");
System.out.print(s.getDepartment()+"\n");
dao.remove(s);
```

```
}

}
```

OUTPUTS:

```
data inserted successfully
data inserted successfully
data inserted successfully
data inserted successfully
data inserted successfully
data inserted successfully
data inserted successfully
data inserted successfully
data inserted successfully
data inserted successfully
data inserted successfully
data fetched successfully
```

```
1 | Ajay | 230510500 | 2025-09-17 20:23:36.405 | AIML
2 | name 0 | 230510500 | 2025-09-17 20:23:36.648 | CSE
3 | name 1 | 230510501 | 2025-09-17 20:23:36.66 | CSE
4 | name 2 | 230510502 | 2025-09-17 20:23:36.671 | CSE
5 | name 3 | 230510503 | 2025-09-17 20:23:36.682 | CSE
6 | name 4 | 230510504 | 2025-09-17 20:23:36.694 | CSE
7 | name 5 | 230510505 | 2025-09-17 20:23:36.705 | CSE
8 | name 6 | 230510506 | 2025-09-17 20:23:36.716 | CSE
9 | name 7 | 230510507 | 2025-09-17 20:23:36.726 | CSE
10 | name 8 | 230510508 | 2025-09-17 20:23:36.738 | CSE
```

```
data updated successfully
data fetched successfully

updated details :
1 | Ajay | 230510500 | 2025-09-17 20:23:36.405 | AIML

Deleted Record details :
22 | name 9 | 230510509 | 2025-09-18 19:03:31.095 | CSE
data removed successfully
```

	id	department	dob	enrollment_no	name
▶	1	AIML	2025-09-17 20:23:36.405000	230510500	Ajay
	2	CSE	2025-09-17 20:23:36.648000	230510500	name 0
	3	CSE	2025-09-17 20:23:36.660000	230510501	name 1
	4	CSE	2025-09-17 20:23:36.671000	230510502	name 2
	5	CSE	2025-09-17 20:23:36.682000	230510503	name 3
	6	CSE	2025-09-17 20:23:36.694000	230510504	name 4
	7	CSE	2025-09-17 20:23:36.705000	230510505	name 5
	8	CSE	2025-09-17 20:23:36.716000	230510506	name 6
	9	CSE	2025-09-17 20:23:36.726000	230510507	name 7
	10	CSE	2025-09-17 20:23:36.738000	230510508	name 8